

# Feasible-Walk Ant Colony Optimization for Pangenome-Guided Assembly

Anonymous Authors

May 29, 2026

## Abstract

Pangenome-guided sequence assembly reconstructs target genomes by traversing population graphs, which reduces reference bias and preserves population variation. The central inference problem is Oriented Tangle Resolution: identify a valid oriented graph walk that agrees with read-derived copy-number estimates. Existing Quadratic Unconstrained Binary Optimization (QUBO) formulations express this decision with time-indexed binary assignments and penalty terms, so graph validity is enforced through the objective rather than through the sample space.

This paper formulates Oriented Tangle Resolution with graph validity as a property of the sample space. Feasible-walk inference constructs legal oriented walks for minigraph-guided assembly and evaluates them with the same copy-number objective used by the original pipeline. We instantiate the formulation with an ACO path finder whose residual-copy heuristic, termination rule, and pheromone feedback operate directly on legal oriented successor choices. A trajectory-trained neural transition prior then biases those choices without changing the feasible action set. On held-out minigraph-pipeline GFAs, the learned prior improves the copy-number optimization objective over zero, shuffled, and random-prior controls, with the strongest gains at short configured budgets. End-to-end assembly results show that feasible-walk search gives the strongest sequence recovery in the reported comparison, while learned guidance provides its clearest benefit at the optimization stage.

## 1 Introduction

Genome assembly remains difficult in repetitive and structurally variable regions. Although de novo assemblers avoid reference bias, they fragment when reads do not bridge repeats; linear-reference workflows, by contrast, can force the reconstruction toward alleles already present in the reference. Sequence-oriented pangenome graphs offer a richer substrate by representing sequence fragments, observed adjacencies, and multiple allelic or structural alternatives in a single graph [1–6]. Once reads from a new sample are mapped to this graph, assembly becomes a constrained path-finding problem: choose a graph-valid walk that explains the read-derived evidence and can be decoded into sequence.

This inference bottleneck is visible in the pangenome-guided assembly pipeline proposed by Cudby et al. [7]. In this workflow, short reads are aligned to a population graph with annotators such as minigraph, GraphAligner, vg Giraffe, or kmer2node [6–9], and the resulting depth profiles are normalized into node-level copy-number estimates. The optimizer’s task is to find an oriented walk that accurately reflects these estimates. The Quadratic Unconstrained Binary Optimization (QUBO) interface encodes this decision with time-indexed binary variables and penalty terms for legal transitions [7, 10]. As a result, graph validity competes with copy-number agreement inside the objective, and search effort can be spent on assignments that do not decode directly into valid biological walks.

The minigraph pangenome is a compressed representation of related genomes. Its construction keeps the graph broadly co-linear with the input genomes, collapses many small local variants into representative or consensus node sequences, and leaves repeats, copy-number changes, inversions, and translocations as graph structures that the path finder must resolve. Because the held-out target is observed through simulated short reads, the inferred walk must satisfy two coupled requirements: it should explain the copy-number surface, and its decoded sequence should survive consensus-quality evaluation against the target. Consequently, structural feasibility must be treated as a hard constraint rather than a soft penalty.

Short-read evidence gives noisy copy-number estimates, so the optimization view remains useful. Pathfinder, the Oatk submodule used as a walk-search comparator in the original benchmark, assigns graph weights and searches for walks that explain them [7, 11]. This prescriptive traversal is effective when coverage is accurate, but it can split an assembly when coverage is imperfect or when unused pangenome nodes remain in the graph. A cost-based objective can instead prefer the walk that best explains the observed copy-number surface. The bottleneck is therefore not optimization itself, but the search space used to represent the assembly decision.

We address this bottleneck by changing the inference formulation while preserving the biological objective and the assembly evaluation. Feasible-walk inference samples valid oriented walks, so graph validity is guaranteed by construction and the path-level copy-number objective can measure assembly evidence directly. In the Ant Colony Optimization (ACO) instantiation [12, 13], each ant starts from an admissible oriented state, repeatedly chooses a legal successor, and terminates at either the artificial end state or the fixed horizon inherited from the QUBO formulation. This formulation makes transition choice the central object of search. The residual-copy heuristic and pheromone update score legal successors by remaining copy-number demand, discourage premature termination before demand is exhausted, and reinforce complete legal walks after path-level evaluation.

The learned extension adds data-driven guidance to this feasible-walk distribution. The transition prior is a graph-conditioned bias over the legal successor set used by the non-neural sampler, so neural guidance changes transition preferences while preserving the feasible action space. The experiments therefore separate the effect of the formulation from the effect of the learned prior. At the optimization stage, the learned prior reduces copy-number energy on held-out minigraph GFAs, and ablation controls show that the improvement is specific to edge-aligned learned logits rather than arbitrary score perturbations. Full assembly then measures how optimization improvements interact with consensus reconstruction.

## 2 Related Work

### 2.1 Classical graph-based sequence assembly

Sequence assembly has long been formulated as inference over a graph whose structure is induced by read overlaps,  $k$ -mers, or repeat relationships. Broad surveys distinguish overlap–layout–consensus, string-graph, and de Bruijn graph assemblers, and emphasize that repeats, uneven coverage, sequencing error, and heterozygosity turn assembly into a graph-disambiguation problem rather than a simple concatenation task [14, 15]. In the overlap-based tradition, string graphs represent irreducible read-overlap relationships after transitive reduction [16]. In the de Bruijn tradition, assembly is transformed from an ordering problem over reads into an Eulerian traversal problem over  $k$ -mer-derived edges [17, 18]; practical short-read assemblers such as Velvet then combine graph compaction, coverage filters, and paired-read information to simplify the graph [19].

Long-read assembly has shifted the balance from short-read repeat ambiguity toward noisy-overlap and haplotype-aware graph resolution. Flye constructs repeat graphs from long, error-prone reads and resolves repeats using graph structure and read support [20]. Hifiasm uses phased assembly graphs to preserve haplotype information in high-fidelity long-read assembly [21], while Verkko combines accurate long reads, ultra-long reads, and haplotype-specific markers to simplify a multiplex de Bruijn graph toward telomere-to-telomere diploid assemblies [22]. These systems show that high-quality assembly depends critically on path extraction from a graph. Our setting differs in that the graph is not built solely from the target reads: it is a population pangenome graph annotated by target-read evidence, and the inference problem is to find an oriented walk whose node visits explain copy-number estimates.

Coverage-aware path extraction also appears in strain-resolved and metagenomic assembly. Haploflow, for example, repeatedly finds coverage-supported paths through a de Bruijn unitig graph and reduces flow along selected paths to recover viral strains from mixed samples [23]. This family of methods demonstrates that path and flow structure can be an effective assembly primitive. The distinction in our problem is that the graph is an external pangenome reference annotated by target-read depth, the output is a single oriented walk through sequence-bearing pangenome states, and the objective penalizes residual node-copy mismatch rather than decomposing an assembly graph into multiple strain flows.

## 2.2 Pangenome graphs and pangenome-guided assembly

Pangenome graphs replace a single linear reference by a graph that represents sequence shared across, and variable among, a population. This representation is motivated by the reference-bias limitations of linear genomes [1] and has been surveyed as a general substrate for read alignment, variant calling, genotyping, visualization, and functional genomics [3, 24]. At human scale, graph pangenomes are now practical reference structures, as illustrated by the Human Pangenome Reference Consortium draft [4]. Toolchains such as vg, minigraph, and PGGB construct or operate on different pangenome graph models, with different tradeoffs between large structural variation, base-level alignment, and scalability [5, 6, 25]. In these graph models, a genome, haplotype, contig, or read alignment is naturally represented as a valid oriented path or walk through a bidirected sequence graph.

Several pangenome methods therefore perform path inference, but with different input-output interfaces. PanGenie uses a hidden Markov model over known haplotype paths and read k-mer counts to infer genotypes across variant bubbles [26]. Recent integer-programming approaches similarly infer pangenome paths for genotyping by optimizing over an expanded graph and read-derived string evidence [27]. These methods reconstruct or genotype haplotypes in a haplotype-resolved reference setting, usually by selecting among known haplotype mosaics or by enforcing a path through an acyclic or expanded graph. Pasa uses population pangenome graphs to scaffold prokaryote assemblies [28], but it orders contigs using pangenome-supported matching scores rather than optimizing a read-depth copy-number walk through a minigraph sequence graph. The closest direct predecessor is the pangenome-guided sequence assembly framework of Cudby et al. [7], which annotates a pangenome graph with read-derived copy-number estimates and formulates assembly as graph traversal optimization through a QUBO interface.

The proposed formulation keeps the biological objective and evaluation boundary of pangenome-guided sequence assembly, but changes the optimization space and search mechanism for Oriented Tangle Resolution. Valid graph walks are a basic object in sequence and variation graphs; here they become the constructive search space for copy-number path finding. Every candidate is a legal oriented pangenome walk, the heuristic measures residual copy-number demand of biological

nodes, and pheromone feedback adapts a stochastic transition distribution over legal successors. This interface differs from HMM or integer-programming haplotype inference, from scaffolding-by-matching, and from QUBO search over time-indexed binary assignments that must be penalized into validity.

### 2.3 Heuristics and meta-heuristics for sequence assembly

Before modern graph assemblers became dominant, the DNA fragment assembly problem was frequently studied as a combinatorial optimization problem over fragment permutations, overlap scores, and contig structure. Genetic algorithms were explored early for the layout phase of fragment assembly [29], and subsequent work compared genetic algorithms, scatter search, CHC, and local-search variants for the fragment assembly problem [30, 31]. Ant-colony approaches were also applied to shotgun fragment assembly: Meksangsouy and Chaiyaratana formulated fragment ordering using ant-colony-system search [32], while Wetcharaporn et al. combined Smith–Waterman overlap computation, ant-colony fragment ordering, and nearest-neighbor contig assembly [33]. Later benchmark and optimization studies standardized comparisons for DNA fragment assembly and explored hybrid metaheuristic overlap–layout–consensus pipelines [34, 35].

These works are important precedents because they treat assembly as a search problem rather than only as a deterministic graph-cleaning pipeline. However, they mostly optimize read or fragment orderings under overlap-based objectives. Ant Colony Optimization itself is a general metaheuristic for discrete graph search in which stochastic constructive agents are biased by pheromone memory and local heuristic information [12, 13, 36]. The assembly space studied here is different: ants traverse an oriented pangenome graph rather than permuting reads or fragments, illegal transitions are absent from the action set, and the terminal objective is copy-number agreement with target-read evidence rather than overlap score alone.

### 2.4 Learned and neural sequence assembly

Learning-based assembly methods have recently begun to target graph resolution directly. GNNome formulates *de novo* assembly as path identification in an assembly graph and uses geometric deep learning to predict graph edges or paths corresponding to reconstructed sequences [37]. This line of work was preceded by an arXiv version showing that a graph convolutional model trained on simulated CHM13-derived graphs could generalize to other human chromosomes and improve contiguity over hand-crafted heuristics on the same graphs [38]. DipGNNome extends the geometric-learning view toward diploid assembly by combining haplotype-aware assembly graphs, supervised edge labels, and beam-search decoding [39]. Reinforcement-learning approaches have also been studied for genome assembly, but recent Q-learning analyses report poor scalability and difficulty obtaining competitive assembly quality in larger environments [40].

The learned prior is related to neural assemblers in that it learns graph-local decisions for path extraction, but the input distribution and inference interface are different. GNNome and DipGNNome operate on *de novo* assembly graphs built from the target reads and learn to decode contigs from those graphs. Here, the graph is a minigraph pangenome with read-depth-derived copy numbers, and the learned model enters as a transition prior inside a feasible-walk optimizer. The neural component ranks legal adjacencies rather than defining the graph itself, so every guided rollout remains a valid oriented walk evaluated through the same objective and downstream pangenome-guided assembly parser.

## 2.5 Learning-guided combinatorial search

The proposed learned transition prior also connects to neural combinatorial optimization and learning-guided search. Pointer networks and reinforcement-learning formulations established that neural policies can construct solutions to routing-like combinatorial problems [41, 42]. Attention models, multiple-start policy optimization, and simulation-guided beam search further improved neural construction and decoding for routing benchmarks [43–45]. More generally, graph neural networks have been used to learn heuristics over graph-structured optimization states [46–49]. Search-guided learning has also been successful in domains where a learned policy or value function biases a symbolic search process rather than replacing it outright [50–54].

The learned ACO variant follows the same hybrid principle: a neural model shapes a classical search distribution, while the classical search procedure maintains feasibility, stochastic exploration, and adaptive memory. It is closest in spirit to dynamic neural guidance for ACO [55], but the biological problem imposes constraints absent from routing benchmarks. A transition is useful only if it is a legal oriented pangenome adjacency and if repeated traversal of its biological node improves copy-number agreement. The learned signal is therefore an edge-level prior aligned with the feasible-walk structure of pangenome-guided sequence assembly.

## 3 Pangenome-Guided Assembly as Feasible-Walk Inference

### 3.1 Assembly pipeline and inference object

The assembly instance begins with two inputs: a Graphical Fragment Assembly (GFA) pangenome and short reads from a target sample [6, 56]. In the GFA, segment records store DNA strings while link records describe adjacencies observed in the population graph. The target sample supplies reads rather than a known path. In the synthetic benchmark, related haploid genomes are generated first; one subset builds the pangenome, while held-out genomes are shredded into reads so that the final assembly can be compared with known truth.

Read-to-graph annotation turns these raw inputs into the weighted graph seen by the path-finding algorithm. The original benchmark studies minigraph, GraphAligner, vg Giraffe, and kmer2node as annotation sources [6–9]. These sources differ in whether they supply read-depth evidence, path-alignment evidence, or k-mer support. The experiments here use the minigraph-pipeline slice, where `SC:f` depth tags are normalized into node copy-number estimates before optimization. The resulting input is therefore not a bare graph, but an annotated pangenome: sequence-bearing nodes, graph adjacencies, node lengths, and a nonnegative copy-number surface estimating how many times each biological segment should be traversed by the target genome.

The experimental intervention is the path-finding stage. Of the six standard assembly stages—graph construction, read simulation, mapping, normalization, path-finding, and evaluation—the proposed solver replaces path finding while upstream annotations and downstream consensus metrics remain unchanged. This comparison isolates the inference formulation from changes to graph construction or sequence evaluation.

The path-finding stage receives a graph that may contain unused population nodes, repeated sequence, collapsed local variation, and noisy node depths. If annotated copy numbers are treated as hard traversal instructions, a zero- or low-coverage node can split the output even when a nearby path through that node would produce a better candidate sequence. Treating copy-number agreement as a cost gives the solver room to prefer a single coherent walk through a noisy region when the estimated copy-number surface is imperfect.

Path-finding output is evaluated both as an optimization solution and as an assembled se-

quence. To obtain sequence-level metrics, the oriented node list is converted into a Graphical mApping Format (GAF)/path representation [6], node sequences are concatenated with reverse complementation when necessary, and the candidate sequence is aligned to the known synthetic truth. Coverage measures how much of the true target is recovered; used sequence measures how much of the candidate is supported by the truth. Contigs and N50 describe continuity, breaks count false joins or large path errors, and indels, difference blocks, and identity measure finer sequence accuracy. Because the downstream parser is part of the evaluation pipeline, an improved path objective is meaningful only when it also improves these assembly-quality metrics.

### 3.2 Annotated graph problem

The input is an annotated GFA-like pangenome graph

$$G = (V, E, w, L, s), \quad (1)$$

where  $V$  is the set of biological sequence nodes,  $E$  is the set of graph adjacencies,  $w : V \rightarrow \mathbb{R}_{\geq 0}$  gives the normalized copy-number estimate derived from read-depth tags,  $L(v)$  is node length, and  $s(v)$  is the DNA sequence stored in node  $v$ . When read or k-mer evidence supplies edge traversal counts, we denote the corresponding oriented count by  $a(i, j)$ . The solver output is not a subset of nodes. It is an ordered walk through graph states, decoded into DNA by concatenating the corresponding node sequences with reverse complementation when needed.

The un-oriented form of the problem asks for a walk  $W$  whose visit count  $\#_W(v)$  matches the estimated copy number  $w(v)$ . Following the pangenome-guided assembly formulation of Cudby et al. [7], this copy-number path problem is Tangle Resolution:

$$J_{\text{TR}}(W) = \sum_{v \in V} (\#_W(v) - w(v))^2. \quad (2)$$

This objective captures the cost-based path-finding idea, but it does not yet account for the double-stranded nature of genomic sequence.

Oriented Tangle Resolution is the corresponding problem on a double-stranded sequence graph. For every biological node  $v$ , the solver creates two oriented states  $v_+$  and  $v_-$ . The oriented graph is

$$\tilde{G} = (\tilde{V}, \tilde{E}), \quad \tilde{V} = \{v_+, v_- : v \in V\} \cup \{\text{end}\}. \quad (3)$$

Every GFA link induces a directed transition together with its reverse-complement mate. To allow a walk to terminate before the fixed horizon, the augmented graph includes the artificial end state end. The horizon follows the same copy-number scaling used by the assembly benchmark:

$$T = \max \left\{ \left\lfloor \kappa \sum_{v \in V} w(v) \right\rfloor, 1 \right\}, \quad (4)$$

with the same horizon scale  $\kappa$  used by the assembly benchmark.

Given a walk  $W$ , let  $c_W(v)$  be the number of visits to biological node  $v$ . In the oriented representation,

$$c_W(v) = \#_W(v_+) + \#_W(v_-). \quad (5)$$

The Oriented Tangle Resolution objective is the node-copy loss

$$J_V(W) = \sum_{v \in V} (c_W(v) - w(v))^2. \quad (6)$$

The feasible-walk solver uses Eq. (6) as its path-finding target. Oriented Tangle Resolution is the reported scope because it preserves the double-stranded biological structure while avoiding the allele-assignment ambiguity of Diploid Tangle Resolution. Diploid and polyploid extensions replace the single walk by multiple walks whose combined oriented visit counts explain the same copy-number surface. The experiments here use the single-walk haploid setting evaluated in the minigraph assembly comparison.

### 3.3 QUBO interface for Oriented Tangle Resolution

After decoding, candidate walks are compared under the same Oriented Tangle Resolution objective and downstream assembly parser used to score paths extracted from the QUBO interface. This compatibility keeps GAF conversion, sequence extraction, and consensus-quality evaluation fixed while removing invalid-transition repair from the solver.

The QUBO interface converts Oriented Tangle Resolution into a time-indexed binary assignment. For each time  $t \in \{1, \dots, T\}$  and each oriented state  $u \in \tilde{V}$ , the encoding introduces a binary variable

$$x_{t,u} = \begin{cases} 1, & \text{state } u \text{ is selected at time } t, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

A valid walk requires exactly one active state at each time and a legal graph transition between consecutive active states. Let  $\mathcal{E}_{\text{end}}$  be the legal transition set after adding termination edges into end and the self-loop (end, end), which lets a completed walk remain at the end state. The QUBO objective combines three terms:

$$C_1(x) = \Lambda_1 \sum_{t=1}^T \left( \sum_{u \in \tilde{V}} x_{t,u} - 1 \right)^2, \quad (8)$$

$$C_2(x) = \Lambda_2 \sum_{t=1}^{T-1} \sum_{\substack{u,v \in \tilde{V} \\ (u,v) \notin \mathcal{E}_{\text{end}}}} x_{t,u} x_{t+1,v}, \quad (9)$$

$$C_3(x) = \sum_{v \in V} \left( \sum_{t=1}^T (x_{t,v_+} + x_{t,v_-}) - w(v) \right)^2. \quad (10)$$

Here  $C_1$  enforces one selected oriented state per time step,  $C_2$  penalizes illegal transitions, and  $C_3$  is the oriented copy-number loss. Because the benchmark horizon scales with the total estimated copy number, the walk can revisit nodes when doing so improves global agreement. The virtual end state allows early termination without forcing unused positions to carry sequence.

The QUBO solved by MQLib or other binary optimizers [10] is the weighted sum

$$C_{\text{QUBO}}(x) = C_1(x) + C_2(x) + C_3(x), \quad (11)$$

up to the standard conversion into a symmetric quadratic matrix. Decoding a feasible assignment gives the oriented state sequence  $(u_1, \dots, u_T)$ , with trailing end states removed before GAF/path conversion and sequence extraction. The same decoded path-level copy-number loss in Eq. (6) is then used to compare candidates produced by the QUBO interface and by our feasible-walk sampler.

Although the time-indexed representation gives the original MQLib backend [10] a standard input format, it also changes the search problem. The oriented encoding uses  $(2|V| + 1)T$  binary

variables for a single walk, and the diploid form introduces a second time-indexed assignment. A binary optimizer therefore explores a large assignment space in which many assignments are not valid paths until penalties push them toward feasibility. Penalty weights must be large enough to enforce constraints, yet not so large that copy-number differences become numerically invisible. Feasible-walk inference moves the validity constraint from the objective into the sample space: while the copy-number objective and downstream parser are unchanged, every candidate is a legal oriented walk at construction time.

The reformulation also exposes a direct route for edge-aware evidence, an important limitation of the original QUBO route. Annotation pipelines can produce edge support when reads or k-mers traverse from one node to another, and such evidence can help distinguish orientation choices that visit the same nodes but use different adjacencies. If  $e_W(i, j)$  counts how many times  $W$  traverses oriented edge  $(i, j)$ , an edge-copy loss can be added as

$$J_E(W) = \sum_{(i,j) \in \tilde{E}} (e_W(i, j) - a(i, j))^2, \quad (12)$$

and the terminal score becomes  $J(W) = J_V(W) + \lambda_E J_E(W)$ . In a time-indexed QUBO,  $e_W(i, j)$  corresponds to products  $x_{t,i}x_{t+1,j}$ ; squaring summed edge residuals then creates higher-order interactions unless auxiliary variables are introduced. In ACO, by contrast, the same quantity is a counter over sampled transitions. The main minigraph comparison keeps the node-copy objective to match the original MQLib comparison, while the edge objective identifies how feasible-walk inference can incorporate transition-level read evidence without changing the candidate representation.

**Feasibility property.** For any ant trajectory  $W = (u_0, \dots, u_\ell)$  produced by the sampler, each nonterminal transition  $(u_t, u_{t+1})$  belongs to  $\tilde{E}$  and each state belongs to  $\tilde{V}$ . The trajectory is therefore a valid oriented walk by construction. The optimization objective can still score poor walks, overlong walks, or walks with copy-number mismatch, but it no longer has to repair illegal graph transitions. Feasibility is enforced by the transition kernel; objective quality remains measured by the same copy-number loss and assembly evaluation.

## 4 Feasible-Walk Ant Colony Optimization for Oriented Tangle Resolution

### 4.1 Ant construction

The feasible-walk sampler constructs a batch of complete legal walks. Let  $S(u)$  be the set of legal successors from oriented state  $u$ , including end when termination is allowed. At step  $t$ , an ant at state  $u_t$  samples  $u_{t+1} \in S(u_t)$  according to

$$p(u_{t+1} = v \mid u_t, c_t) = \frac{\tau_{u_t v}^{\alpha_\tau} \eta(v, c_t)^{\beta_\eta}}{\sum_{z \in S(u_t)} \tau_{u_t z}^{\alpha_\tau} \eta(z, c_t)^{\beta_\eta}}, \quad (13)$$

where  $\tau_{uv}$  is pheromone,  $c_t$  is the current node-visit count vector, and  $\eta(v, c_t)$  is a residual-copy heuristic. The heuristic gives high score to successors whose underlying biological node still has unmet copy-number demand:

$$\eta(v, c_t) = \begin{cases} 0.01 + \max(0, w(b(v)) - c_t(b(v))) \left(1 + \frac{\sqrt{1+L(b(v))}}{10}\right), & v \neq \text{end}, \\ \eta_{\text{end}}(c_t), & v = \text{end}, \end{cases} \quad (14)$$



where  $b(v)$  maps an oriented state to its biological node. The end-state score is

$$\eta_{\text{end}}(c_t) = \begin{cases} 1, & \sum_{x \in V} \max(0, w(x) - c_t(x)) = 0, \\ 0.01, & \text{otherwise.} \end{cases} \quad (15)$$

The end state therefore receives a high score only after residual demand is exhausted; otherwise it remains available but discouraged.

Legal-successor sampling gives the assembly solver two properties. Every ant path can be converted back into a graph path without repair. Residual-copy scores also change as the ant consumes node demand, allowing the same edge to be attractive early in a walk and unattractive after repeated visits to its biological node.

## 4.2 Pheromone feedback

Pheromone feedback turns batches of legal walks into an adaptive search distribution. After each ant batch, decoded path-level scores identify elite traces, which then update pheromone. Evaporation is applied as

$$\tau_{uv} \leftarrow \max\{(1 - \rho)\tau_{uv}, \epsilon\}, \quad (16)$$

where  $\rho$  is the evaporation rate and  $\epsilon$  prevents zero-probability transitions. The best quarter of the ants deposit additional pheromone on their traversed edges:

$$\tau_{uv} \leftarrow \tau_{uv} + \Delta_k \quad \text{for each edge } (u, v) \text{ in elite ant } k. \quad (17)$$

The deposit  $\Delta_k$  is scaled by the ant’s energy relative to the worst energy in the batch, so lower-energy ants reinforce their edges more strongly. Evaporation and residual demand preserve exploration.

**Adaptive feasible-walk distribution.** The sampler maintains an incumbent walk  $W^*$  and a pheromone field initialized to  $\tau_e = 1$  on every legal transition. During one inference update, it draws  $M$  legal walks from Eq. (13), evaluates them with the decoded path-level objective, stores any lower-energy incumbent, and shifts the transition distribution through evaporation and elite-trace deposition. Additional inference compute increases the number of feedback cycles in which sampled walk quality can reshape the transition distribution. The minimum-update requirement guarantees at least one such feedback cycle.

## 4.3 Compute-quality tradeoff

Sampled walks and pheromone updates define the compute-quality tradeoff of the feasible-walk formulation. Pangenome graph builders produce subgraphs with different sizes and branching structures, so the relevant question is how effectively additional inference compute improves already-valid oriented walks rather than how often invalid assignments can be repaired after optimization.

# 5 Learned Transition Priors

## 5.1 From static guidance to dynamic transition shaping

The learned signal must be aligned with the search state in which it is used. Although a fixed heatmap can help at initialization, the feasible-walk distribution changes as pheromone accumulates and as the incumbent improves. The proposed policy adapts the DyNACO idea of dynamic neural

guidance [55]: training uses decisions sampled inside the iterative search trajectory rather than a one-shot static heatmap.

The pangenome instantiation preserves the legal-walk sampler, residual-copy heuristic, and pheromone dynamics before adding a graph neural edge prior to the transition score. The neural policy supplies edge-level guidance to an adaptive feasible-walk environment whose state changes through pheromone updates and incumbent improvements. Feasible walks replace TSP tours and copy-number loss replaces route length; the learned object retains the role of dynamic neural guidance as an intervention on the transition distribution.

## 5.2 Trajectory-trained transition prior

The learned prior is trained at the same temporal granularity at which it will influence search: a neural decision shapes a block of feasible-walk sampling, and pheromone feedback changes the search state before the next neural decision. Each outer update observes the oriented graph instance and emits a vector of edge logits once. The inner environment then runs several ACO batches with those logits held fixed while pheromone changes after every batch. The neural action is therefore a fixed-duration macro-action that biases a block of legal successor decisions before stochastic sampling, path scoring, and elite feedback produce the next macro-state.

For training graph  $G$ , outer update  $h$ , and inner batch  $s$ , the rollout state can be written as

$$\mathcal{S}_{h,s} = (G, w, \tau^{(h,s)}, c_t, u_t, T), \quad (18)$$

where  $\tau^{(h,s)}$  is the pheromone vector at the start of the batch,  $u_t$  and  $c_t$  are the current ant state and visit counts collapsed across both orientations during construction, and  $T$  is the horizon. At the outer update, the learned action is an edge-logit vector

$$z_\theta(G, w) = \{r_\theta(u, v; G, w) : (u, v) \in \mathcal{E}_{\text{end}}\}. \quad (19)$$

The edge-logit vector is not recomputed from the live pheromone field inside the inner rollout. Instead, trajectory-aware training exposes the prior to changing pheromone states across rollout blocks, while deployment holds the current edge prior fixed within each block. Adaptation inside a block comes from pheromone feedback and the residual-copy heuristic; adaptation across blocks comes from the next neural prior update.

Deployment combines the held prior with the current ACO state at every legal successor decision. For an ant at state  $u_t$  with legal successor  $v$ , the deployed score is

$$\log s(v \mid u_t, c_t, h, s) = \alpha_\tau \log \tau_{u_t v}^{(h,s)} + \beta_\eta \log \eta(v, c_t) + \gamma r_\theta(u_t, v; G, w). \quad (20)$$

The transition distribution is the softmax of Eq. (20) over  $v \in S(u_t)$ . Setting  $\gamma = 0$ , or using a zero prior, recovers the non-learned ACO variant. The learned component is therefore a bias over legal transitions, not a mechanism that can create infeasible paths.

At deployment,  $z_\theta(G, w)$  is computed once for the outer update. Each of the `mini_H` inner batches samples  $n_{\text{ants}}$  feasible walks, scores them with the decoded path-level objective, updates pheromone from elite traces, and retains the incumbent. Evaluation uses the same loop, with the learned checkpoint supplying the held prior and pheromone initialized locally for each instance.

## 5.3 Graph neural edge prior

The graph neural prior estimates which legal transitions remain useful given both local construction state and the global copy-number surface. Its representation must distinguish residual demand from

over-use, recognize the current oriented state, and compare legal successors in the context of graph density and remaining horizon. We encode these signals with node features for copy number, visit count, residual demand, length, orientation, current-state status, zero-copy status, and local degree; edge features for source-target incidence, end-state structure, artificial start/end transitions, self-transitions, endpoint copy numbers, and target residual state; and global features for prefix depth, total copy-number mass, residual/over/under-use, and graph density.

The online trainer builds the prior tensor from the start-state graph tensor, with zero visit counts, the artificial start state, and the QUBO-derived walk horizon. The model emits one logit for every legal edge in the augmented transition set. During replay and sampling,  $\eta(v, c_t)$  changes with the ant prefix and pheromone changes across inner batches, while the neural logit for a source-target edge remains the held prior value produced at the outer update.

Node and edge features are linearly embedded before passing through a stack of edge-aware message-passing layers. Within each layer, gated source-node messages pass through edge states; node embeddings are updated by a residual normalized transformation; edge embeddings are updated from the source embedding, target embedding, and previous edge state. A scalar edge head decodes each legal transition logit from the edge state, source embedding, target embedding, and global features. The reported configuration uses 2 message-passing layers, hidden dimension 16, AdamW optimization [58], and gradient clipping.

The learned prior enters a search distribution that already contains pheromone feedback and the residual-copy heuristic. Its role is to supply graph-contextual preference among legal successors: it can reinforce a low-pheromone edge whose successor helps satisfy residual demand, suppress an over-reinforced edge that repeatedly creates copy-number mismatch, or separate alternatives with similar local heuristic scores but different graph context. The paired comparison between feasible-walk ACO and learned-prior ACO isolates this term in Eq. (20); graph instances, copy-number preprocessing, sampled-walk count, objective, and consensus evaluation remain fixed.

## 5.4 Online rollout training

The learned prior is trained online from feasible-walk ACO rollouts; no target paths or offline labels are precomputed. For each training graph, pheromone starts at one on every legal transition. At outer update  $h$ , the trainer computes a detached prior  $z_{\theta_{\text{old}}}(G, w)$  and uses it to sample mini\_H inner ant batches. Before each inner batch, the trainer stores the current pheromone vector. The C++ sampler records each ant trace as start offsets and chosen edge ids; after batch scoring, elite traces update pheromone before the next batch is sampled.

Trajectory replay then evaluates the sampled decisions under the current model. Replay recomputes the log-probability of each chosen edge using the stored pheromone snapshot, the current model’s held prior, and the dynamic residual heuristic reconstructed from the prefix visit counts. For an ant walk  $W_a = (u_0, \dots, u_\ell)$ , the replayed log-probability is

$$\log p_\theta(W_a \mid \tau, z_\theta) = \sum_{t=0}^{\ell-1} \log \frac{\exp(\log s(u_{t+1} \mid u_t, c_t, h, s))}{\sum_{v \in S(u_t)} \exp(\log s(v \mid u_t, c_t, h, s))}. \quad (21)$$

The trajectory score is length-normalized by the number of stochastic decisions in the trace before forming the batch loss. If ant  $a$  in batch  $B$  produces walk  $W_a$  with energy  $J(W_a)$ , the batch-centered cost advantage is

$$A_a = \bar{J}_B - J(W_a), \quad \bar{J}_B = \frac{1}{|B|} \sum_{b \in B} J(W_b). \quad (22)$$

The minimized replay loss is

$$\mathcal{L}(\theta) = \frac{1}{|B|} \sum_{a \in B} (J(W_a) - \bar{J}_B) \log p_{\theta}(W_a \mid \tau, z_{\theta}), \quad (23)$$

with stop-gradient costs. Under gradient descent, lower-energy walks have negative centered cost and therefore receive increased probability, while higher-energy walks are suppressed. The objective is a REINFORCE trajectory replay loss with batch baselines and gradient clipping [57]. Pheromone is treated as rollout-local search state; the transferable learned object is the neural transition prior.

The training objective is trajectory-aware: the policy is optimized on walks sampled inside the pheromone loop, not on a single pheromone-free construction step. Pheromone concentration, elite traces, and residual-copy patterns change during search, so the same graph can induce different training signals at different outer updates. The learned prior is trained to complement ACO’s adaptive memory: neural logits provide graph-contextual transition bias, while pheromone feedback supplies rollout-local adaptation.

The training distribution follows the minigraph assembly pipeline. Each training instance comes from the same upstream simulation used for evaluation: synthetic haplotypes define a population graph, a held-out genome is shredded into simulated reads, and minigraph writes an annotated query GFA with `SC:f` read-depth tags. Before training or evaluation, the tags are normalized into node copy numbers using the benchmark conversion with minimum copy number 0.45, integer mode, offset 0.4, and depth divisor 5.0. The learned policy therefore sees the same kind of normalized copy-number surface used by the minigraph benchmark.

The data schedule separates the online training stream, the fixed validation set, and the held-out test sets. Training draws from an expanding stream of minigraph-pipeline instances. Validation uses 16 fixed GFAs from disjoint seeds for model selection, whereas final optimization comparisons use separate held-out minigraph instances. This exposes the policy to varied graph and copy-number surfaces during learning while keeping model selection distinct from paired test comparisons.

The objective-evaluation path materializes a dense time-indexed QUBO matrix, so QUBO-stage supervision uses a scale guard whose memory demand grows with both node count and horizon. The guard filters training and validation GFAs by segment count and by the horizon estimated after benchmark-style copy-number normalization. This computational constraint applies to the supervised training subset; the target assembly distribution remains the minigraph-pipeline distribution. Dense QUBO materialization limits training graphs to at most 30 segments, and the held-out optimization evaluation therefore tests cross-scale generalization of the learned transition prior rather than same-scale supervised tuning.

## 6 Evaluation Interface

The method changes the path-finding stage of the pangenome-guided assembly pipeline. Upstream graph generation, read simulation, read-to-graph annotation, and copy-number estimation remain unchanged within each evaluated condition. Downstream, after the selected walk is converted to GAF/path output and the path sequence is extracted, the same consensus and assembly-quality evaluation computes coverage, used sequence, contig count, breaks, indels, differences, and identity. Coverage measures truth recovery, used sequence measures support for the candidate assembly, contigs and N50 measure continuity, breaks measure false joins, and indels, difference blocks, and identity measure sequence-level accuracy. This boundary tests two inference spaces for Oriented Tangle Resolution: penalty-constrained binary assignments and feasible oriented walks.

Table 1: Learned-prior protocol for trajectory-aware training and paired comparison.

| Quantity                                    | Setting                                                             |
|---------------------------------------------|---------------------------------------------------------------------|
| Training epochs                             | 10                                                                  |
| Training steps per epoch                    | 16                                                                  |
| Training source                             | Online minigraph-pipeline GFA stream                                |
| Validation source                           | 16 fixed minigraph-pipeline GFAs from held-out seeds                |
| Copy-number source                          | Paper-normalized copy numbers from GFA <code>SC:f</code> depth tags |
| Feasibility guard                           | At most 30 segments and estimated horizon at most 300               |
| Online guidance steps $H$                   | 10                                                                  |
| Inner steps mini_H                          | 10                                                                  |
| Total ant batches $H \times \text{mini\_H}$ | 100                                                                 |
| Ants $n_{\text{ants}}$                      | 32                                                                  |
| GNN depth / hidden units                    | 2 / 16                                                              |
| Optimizer / learning rate                   | AdamW [58] / $5 \times 10^{-4}$                                     |
| Gradient clipping                           | 1.0                                                                 |
| Pheromone exponent $\alpha_\tau$            | 1.0                                                                 |
| Residual heuristic exponent $\beta_\eta$    | 1.0                                                                 |
| Evaporation $\rho$                          | 0.1                                                                 |
| Elite fraction                              | 0.25                                                                |
| Neural prior weight $\gamma$                | 1.0                                                                 |

The data split separates paired evaluation from online learning. A fixed minigraph corpus supports solver comparisons on identical graph instances. An online rollout stream supplies training graphs from the same minigraph pipeline, a 16-instance validation set supports model selection, and held-out test sets support the reported optimization and assembly comparisons.

For assembly comparisons, Pathfinder preprocessing follows the original benchmark’s QUBO subgraph construction whenever that preprocessing mode is part of the evaluated minigraph solver condition [7]. In that end-to-end Pathfinder-graph condition, the original assembly helper supplies subgraph copy-number conversion. For learned-prior training, validation, and standalone QUBO-stage comparisons, the same minigraph pipeline stops after annotated query GFAs are produced. Their `SC:f` depth tags are normalized into benchmark-style copy numbers and exposed as training-stream, validation, or held-out optimization instances. The original benchmark also studies GraphAligner, vg Giraffe, and kmer2node annotation routes; the learned-prior experiments reported here use the minigraph-pipeline slice so the model sees one consistent annotated-GFA distribution. Minigraph-pipeline graphs contain read-depth tags, mapping noise, and preprocessing-induced subgraphs absent from simple line-and-shortcut generators, so the learned prior is trained on the same kind of copy-number surface used by the minigraph assembly benchmark while the end-to-end comparison preserves the original assembly route.

## 7 Experiments

### 7.1 Evaluation protocol

The evaluation follows the two contribution structure. The first ablation isolates legal-walk sampling as the non-learned carrier for the assembly formulation. Held-out optimization instances then measure the effect of edge-level guidance by comparing learned-prior ACO against the same feasible-walk ACO distribution and against non-neural baselines. A learned-prior ablation varies

Table 2: Feasible-walk comparison under the original assembly pipeline. Feasible-walk ACO is the proposed legal-walk sampler without the learned transition prior. Metrics follow the original consensus-quality parser. Higher Covered, Used, and Identity are better; lower Contigs, Breaks, Indels, and Diffs are better.

| Graph     | Solver            | Time | Covered | Used  | Contigs | Breaks | Indels | Diffs | Identity |
|-----------|-------------------|------|---------|-------|---------|--------|--------|-------|----------|
| Minigraph | MQLib             | 5    | 47.16   | 99.98 | 4.00    | 0.00   | 0.67   | 0.00  | 98.49    |
| Minigraph | Feasible-walk ACO | 5    | 59.06   | 94.59 | 5.00    | 0.67   | 0.00   | 0.00  | 99.31    |

the prior source and neural weight, testing whether the neural signal carries edge-specific information beyond zero, shuffled, or random edge logits. Full assembly runs pass the feasible-walk solvers through the original sequence parser to measure how intermediate optimization quality becomes assembly-level recovery and structural correctness.

All comparisons preserve the biological objective and the evaluation boundary. Upstream graph construction, minigraph read mapping, copy-number normalization, path extraction, and consensus-quality evaluation are shared, so differences come from the inference formulation and its learned transition prior. At the optimization stage, the comparison has three roles. MQLib is the direct external baseline because it is the QUBO heuristic backend used for Oriented Tangle Resolution in the original pangenome-guided assembly interface [7, 10]. Pathfinder is the original walk-search baseline in that pipeline: it combines node and edge weights in an exhaustive graph traversal designed for coverage-annotated assembly graphs [7, 11]. Beam search and A\* provide graph-search controls adapted to the same oriented objective [59, 60]. Feasible-walk ACO provides the non-neural internal variant, derived from ant-colony search [12, 13], and isolates the learned guidance layer.

The end-to-end baseline set follows the task definition of pangenome-guided sequence assembly: the solver receives an annotated minigraph instance and returns a copy-number-scored oriented walk for the same consensus parser. Related pangenome tools use different input-output interfaces. Pasa uses population pangenome graphs for scaffolding prokaryote assemblies [28]; PanGenie and integer-programming pangenome inference methods perform genotyping or haplotype inference from known graph structure and read evidence [26, 27]; and neural graph decoders such as GNNome and DipGNNome operate on *de novo* assembly graphs [37, 39]. The direct empirical baseline is therefore the original QUBO/MQLib route under the same pangenome-guided assembly pipeline. At full assembly scope, feasible-walk ACO and learned-prior ACO are evaluated through the consensus parser on minigraph instances, and the assembly results measure downstream transfer from path optimization to sequence recovery.

## 7.2 Ablation: feasible-walk path finding

In the reformulation check, feasible-walk ACO and MQLib share minigraph graph generation, copy-number normalization, path extraction, and the consensus parser. The comparison changes where feasibility is enforced: through QUBO penalties or through legal-walk construction.

Assembly instance construction remains fixed while the inference space changes. The comparison uses three seeds with one test sequence per seed, Pathfinder graph preprocessing, and a five-second path-finding call in which the feasible-walk sampler draws 8 ants and performs at least one pheromone-update iteration.

Table 2 isolates the effect of moving feasibility from objective penalties into the sampling space. Relative to MQLib, feasible-walk ACO increases coverage on minigraph-guided assemblies while trading off used sequence and continuity. This ablation shows that direct legal-walk construction

Table 3: Measured wall-clock optimization-stage comparison on 10 held-out minigraph test instances. The table includes the direct QUBO/MQLib baseline, graph-search controls adapted to the same oriented objective, and feasible-walk ACO variants. LP-ACO denotes learned-prior ACO. Energy, gap, and runtime are means over instances. Lower energy, lower gap, and lower runtime are better; wins count instances tied for best energy at that budget. Runtime includes the solver call plus shared QUBO construction and wrapper overhead.

| Budget | Solver      | Mean energy | Gap    | Wins | Runtime (s) |
|--------|-------------|-------------|--------|------|-------------|
| 5      | LP-ACO      | 40.78       | 0.00   | 10   | 277.24      |
| 5      | A*          | 42.12       | 1.34   | 0    | 19.51       |
| 5      | FW-ACO      | 44.20       | 3.42   | 0    | 16.81       |
| 5      | Beam search | 44.51       | 3.73   | 0    | 19.21       |
| 5      | MQLib       | 193.03      | 152.26 | 0    | 25.05       |
| 10     | LP-ACO      | 40.78       | 0.13   | 5    | 285.53      |
| 10     | Beam search | 41.22       | 0.57   | 5    | 33.16       |
| 10     | A*          | 42.12       | 1.47   | 0    | 35.02       |
| 10     | FW-ACO      | 44.34       | 3.69   | 0    | 17.69       |
| 10     | MQLib       | 149.22      | 108.57 | 0    | 41.62       |
| 180    | LP-ACO      | 40.78       | 0.13   | 5    | 548.39      |
| 180    | Beam search | 41.22       | 0.57   | 5    | 35.96       |
| 180    | A*          | 42.12       | 1.47   | 0    | 545.26      |
| 180    | FW-ACO      | 44.97       | 4.32   | 0    | 185.55      |
| 180    | MQLib       | 89.78       | 49.12  | 0    | 552.79      |

can improve the assembly objective before learned guidance is added, although the continuity tradeoff leaves room for a transition prior to reshape the feasible-walk distribution.

### 7.3 Optimization quality from learned transition priors

The learned-prior comparison starts from minigraph-pipeline GFAs with `SC:f` depth tags. After the tags are converted into benchmark-style copy numbers, the graphs are transformed into oriented optimization instances; held-out test graphs remain separate from the 16-instance validation set. Feasible-walk ACO and learned-prior ACO share the feasible-walk sampler, residual heuristic, pheromone update, objective, graph distribution, and output representation, leaving the graph-neural transition prior as the isolated difference.

Under matched objective evaluation, the learned-prior comparison tests whether edge-level guidance changes the quality reached by the same feasible-walk search distribution. The comparison uses 10 held-out minigraph query GFAs and evaluates MQLib, feasible-walk ACO, learned-prior ACO, beam search, and A\* through the same Oriented Tangle Resolution objective. These comparators cover the original QUBO backend, adapted bounded-width heuristic search, adapted classical best-first graph search, and the non-neural version of the proposed ACO sampler. The reported runtime is measured end to end for the solver wrapper, including shared QUBO construction overhead. The learned-prior solver uses the trajectory-trained ACO setting from Table 1.

Table 3 shows that the learned transition prior shifts the feasible-walk distribution toward lower-copy-loss walks on held-out minigraph instances. At the shortest budget, learned-prior ACO wins all 10 instances and lowers mean energy from 42.12 for A\* and 44.20 for feasible-walk ACO to 40.78. At 10 seconds and above, learned-prior ACO and beam search split the instance wins, with learned-prior ACO retaining the lowest mean energy. The measured runtime is substantially higher for the current neural implementation because each instance runs the full trajectory-trained ACO loop and repeated neural-guided sampling. These results support learned transition guidance as a solution-quality mechanism, while runtime remains an implementation tradeoff.

Table 4: Learned-prior ablation at the 5 second configured budget on 10 held-out minigraph instances and three stochastic seeds. Rows report 30 evaluations. Gap is relative to the best condition for the same seed and instance. Lower energy and gap are better.

| Prior    | $\gamma$ | Rows | Mean energy | Gap   | Wins |
|----------|----------|------|-------------|-------|------|
| Learned  | 0.25     | 30   | 40.66       | 0.00  | 25   |
| Learned  | 0.50     | 30   | 40.78       | 0.12  | 15   |
| Learned  | 1.00     | 30   | 40.78       | 0.12  | 15   |
| Zero     | 1.00     | 30   | 40.84       | 0.19  | 5    |
| Shuffled | 0.25     | 30   | 44.44       | 3.79  | 0    |
| Random   | 0.25     | 30   | 43.59       | 2.93  | 0    |
| Shuffled | 1.00     | 30   | 72.37       | 31.72 | 0    |
| Random   | 1.00     | 30   | 73.55       | 32.90 | 0    |

#### 7.4 Ablation: learned guidance versus control priors

The prior ablation asks whether the learned component contributes useful graph information or merely perturbs the ACO transition distribution. We evaluate three stochastic seeds on the 10 held-out minigraph optimization instances at the 5 second configured budget, using the same feasible-walk sampler and changing only the prior source and neural weight  $\gamma$ . A zero prior removes neural information. A shuffled prior preserves the learned logit multiset but breaks edge identity. A random prior injects Gaussian edge scores with the same scale as the learned logits.

Table 4 shows that useful guidance is edge-specific and learned. A small learned prior weight gives the best mean energy and wins 25 of 30 seed-instance evaluations. Setting the learned weight to zero recovers the non-neural prior-free behavior. Shuffling or randomizing the prior degrades the search, especially at larger weights, showing that the benefit comes from edge identity rather than from arbitrary score perturbations. The best ablation setting is  $\gamma = 0.25$ , while the default  $\gamma = 1.0$  remains competitive but less robust. Prior strength is therefore a distribution-sensitive parameter of the learned feasible-walk interface.

To evaluate transfer beyond the 73–79 segment held-out set, we also use a second minigraph test slice with mean segment count 100. This scale evaluation uses the same trained checkpoint and the same configured solver limits, but changes the selected GFA instances. Table 5 shows that learned-prior ACO remains the lowest-energy solver on all 12 instances across the tested configured limits. The result supports cross-scale transfer of the learned transition prior at the optimization stage, while preserving the runtime pattern from Table 3: the current neural implementation has higher measured runtime than the non-neural baselines.

#### 7.5 End-to-end assembly results

To test whether intermediate optimization gains translate to actual assembly quality, the seed-matched comparison evaluates minigraph assemblies with the original consensus parser. The comparison uses seeds 00001–00008, with five held-out sequences per seed, for 40 assemblies per complete row. The configured path-finding limits define the assembly conditions. A repeated 10 second learned-prior row uses a different solver seed to measure stochastic stability.

Table 6 shows that downstream assembly transfer is not a simple restatement of QUBO-stage energy. The strongest assembly row is the non-neural feasible-walk ACO at 30 seconds: it improves coverage, used sequence, contig count, and breaks relative to the MQLib 30 second row. Learned-prior ACO gives high used sequence and lower break counts than MQLib in the main run, while coverage varies across solver seeds. These results make feasible-walk ACO the stronger



Table 5: Optimization-stage scale evaluation on 12 held-out minigraph instances with mean segment count 100. All rows use the same trained learned-prior checkpoint. Configured limit is the solver limit supplied to each method; measured runtime includes solver call and shared wrapper overhead. Lower energy and lower gap are better.

| Solver      | Configured limit | Rows | Mean energy | Gap to best | Wins | Measured runtime (s) |
|-------------|------------------|------|-------------|-------------|------|----------------------|
| LP-ACO      | 1                | 12   | 55.95       | 0.00        | 12   | 36.60                |
| FW-ACO      | 1                | 12   | 67.00       | 11.04       | 0    | 11.69                |
| Beam search | 1                | 12   | 79.14       | 23.19       | 0    | 11.66                |
| MQLib       | 1                | 12   | 340.44      | 284.49      | 0    | 19.95                |
| LP-ACO      | 5                | 12   | 55.95       | 0.00        | 12   | 31.99                |
| FW-ACO      | 5                | 12   | 65.79       | 9.84        | 0    | 13.60                |
| Beam search | 5                | 12   | 69.64       | 13.69       | 0    | 13.42                |
| MQLib       | 5                | 12   | 419.31      | 363.35      | 0    | 21.81                |
| LP-ACO      | 10               | 12   | 55.95       | 0.00        | 12   | 29.69                |
| FW-ACO      | 10               | 12   | 67.10       | 11.15       | 0    | 18.00                |
| Beam search | 10               | 12   | 63.66       | 7.70        | 0    | 17.94                |
| MQLib       | 10               | 12   | 557.01      | 501.05      | 0    | 26.12                |

Table 6: Full minigraph assembly comparison through the original consensus parser. LP-ACO denotes learned-prior ACO and FW-ACO denotes feasible-walk ACO without the learned prior. Configured limit is the per-call path-finding time limit, not measured end-to-end runtime. Higher Covered, Used, and Identity are better; lower Contigs and Breaks are better.

| Graph     | Solver      | Configured limit | Seqs | Covered | Used  | Contigs | Breaks | Identity |
|-----------|-------------|------------------|------|---------|-------|---------|--------|----------|
| Minigraph | MQLib       | 30               | 40   | 74.45   | 75.93 | 5.15    | 3.18   | 98.91    |
| Minigraph | Beam search | 30               | 40   | 72.43   | 84.38 | 4.23    | 3.13   | 99.20    |
| Minigraph | FW-ACO      | 30               | 40   | 76.97   | 90.89 | 4.90    | 1.43   | 98.82    |
| Minigraph | LP-ACO      | 5                | 40   | 73.38   | 91.51 | 4.45    | 1.98   | 98.89    |
| Minigraph | LP-ACO      | 10               | 40   | 69.33   | 90.24 | 4.18    | 2.13   | 99.23    |
| Minigraph | LP-ACO      | 10 repeat        | 40   | 68.99   | 81.17 | 5.38    | 4.30   | 99.10    |
| Minigraph | LP-ACO      | 30               | 40   | 70.96   | 89.53 | 4.60    | 2.25   | 99.29    |

assembly-level contribution and position neural guidance as an optimization-stage improvement whose assembly transfer depends on stochastic search behavior and the downstream consensus parser.

## 8 Limitations

The optimization evidence covers held-out minigraph instances in the 73–79 segment range, with an additional 100-segment scale evaluation. The end-to-end evidence covers the reported 40-sequence minigraph assembly comparison. Broader scale claims require repeated timing measurements across additional graph sizes and an evaluation path that separates QUBO construction overhead from solver time. The current learned-prior implementation improves copy-number energy but is slower in measured wall-clock time than beam search and feasible-walk ACO.

The method inherits upstream uncertainty from graph construction, read-to-graph annotation, and copy-number estimation. The feasible-walk sampler chooses among paths present in the pangenome graph, so sample-specific sequence absent from the graph is outside its action space. The residual heuristic may over-favor long high-copy nodes when copy-number estimates are biased, and the reported learned-prior experiments do not test the GraphAligner, vg Giraffe, or kmer2node annotation routes from the broader benchmark. Although learned-prior training uses minigraph assembly instances, the dense-QUBO evaluation path restricts training to a feasible sub-

set of that distribution. Removing this subset restriction requires sparse QUBO construction or a direct walk-energy training path that does not materialize the full time-indexed matrix. Finally, improved copy-number energy does not guarantee better consensus-level assembly metrics; the repeated learned-prior assembly run shows that stochastic stability remains an important target for future work.

## 9 Conclusion

Oriented Tangle Resolution can be solved in the space of feasible oriented walks rather than in a penalty-constrained binary assignment space. Feasible-walk inference makes legal-walk construction the search primitive, and the ACO instantiation provides residual-copy heuristics, termination handling, and pheromone feedback for this pangenome-specific action space. This formulation keeps the copy-number objective and consensus parser of the original pipeline, while avoiding the time-indexed invalid-assignment space used by QUBO/MQLib. A trajectory-trained neural transition prior then supplies graph-conditioned guidance over legal successor choices. The learned prior improves held-out optimization-stage energy and beats zero, shuffled, and random-prior controls, especially at short configured budgets. In full assembly, feasible-walk ACO gives the strongest sequence recovery in the reported comparison, while learned guidance improves the optimization objective and some structural metrics. The resulting picture is a feasible-walk optimization interface for pangenome path finding, with neural transition priors as a data-driven extension whose runtime, annotation-source transfer, edge-evidence integration, and stochastic stability define the next implementation targets.

## References

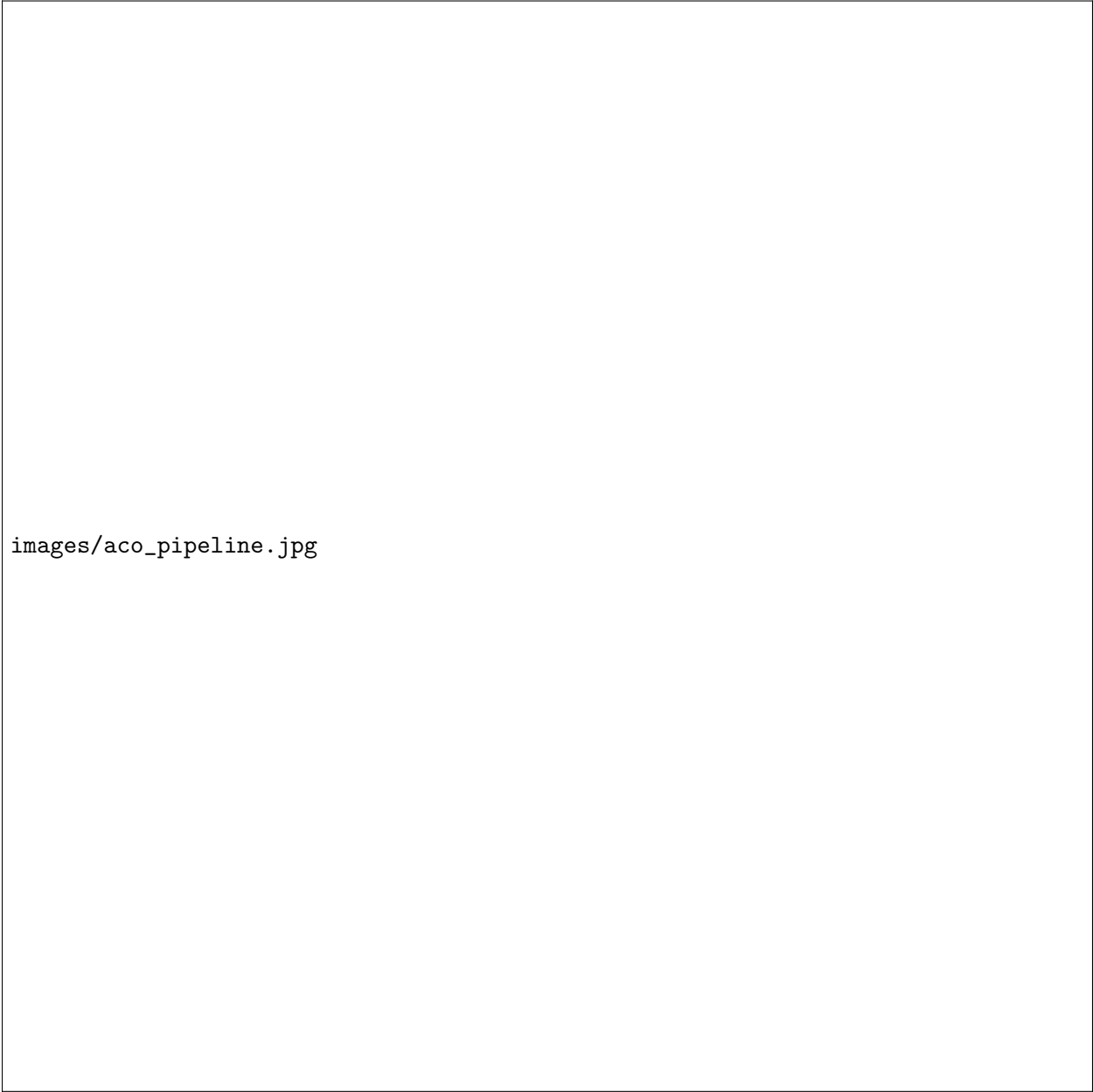
- [1] Sara Ballouz, Alexander Dobin, and Jesse A. Gillis. Is it time to change the reference genome? *Genome Biology*, 20(1):159, 2019. doi: 10.1186/s13059-019-1774-4.
- [2] Valerie A. Schneider, Tina Graves-Lindsay, Kerstin Howe, et al. Evaluation of GRCh38 and de novo haploid genome assemblies demonstrates the enduring quality of the reference assembly. *Genome Research*, 27(5):849–864, 2017. doi: 10.1101/gr.213611.116.
- [3] Chelsea A. Matthews, Nathan S. Watson-Haigh, Rachel A. Burton, et al. A gentle introduction to pangenomics. *Briefings in Bioinformatics*, 25(6):bbae588, 2024. doi: 10.1093/bib/bbae588.
- [4] Wen-Wei Liao, Mobin Asri, Jana Ebler, et al. A draft human pangenome reference. *Nature*, 617:312–324, 2023. doi: 10.1038/s41586-023-05896-x.
- [5] Glenn Hickey, David Heller, Jean Monlong, et al. Genotyping structural variants in pangenome graphs using the vg toolkit. *Genome Biology*, 21(1):35, 2020. doi: 10.1186/s13059-020-1941-7.
- [6] Heng Li, Xiaowen Feng, and Chong Chu. The design and construction of reference pangenome graphs with minigraph. *Genome Biology*, 21(1):265, 2020. doi: 10.1186/s13059-020-02168-z.
- [7] Josh Cudby, James Bonfield, Chenxi Zhou, Richard Durbin, and Sergii Strelchuk. Pangenome-guided sequence assembly via binary optimisation. *Briefings in Bioinformatics*, 27(1):bbag084, 2026. doi: 10.1093/bib/bbag084.
- [8] Mikko Rautiainen and Tobias Marschall. Graphaligner: rapid and versatile sequence-to-graph alignment. *Genome Biology*, 21(1):253, 2020. doi: 10.1186/s13059-020-02157-2.

- [9] Jouni Sirén, Jean Monlong, Xian Chang, et al. Pangenomics enables genotyping of known structural variants in 5202 diverse genomes. *Science*, 374(6574):abg8871, 2021. doi: 10.1126/science.abg8871.
- [10] Iain Dunning, Swati Gupta, and John Silberholz. What works best when? a systematic evaluation of heuristics for max-cut and QUBO. *INFORMS Journal on Computing*, 30(3): 608–624, 2018. doi: 10.1287/ijoc.2017.0798.
- [11] Chenxi Zhou, Max Brown, Mark Blaxter, et al. Oatk: a de novo assembly tool for complex plant organelle genomes. *Genome Biology*, 26(1):235, 2025. doi: 10.1186/s13059-025-03676-6.
- [12] Marco Dorigo and Luca Maria Gambardella. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, 1997. doi: 10.1109/4235.585892.
- [13] Marco Dorigo and Thomas Stützle. *Ant Colony Optimization*. MIT Press, Cambridge, MA, 2004. ISBN 9780262042192.
- [14] Jason R. Miller, Sergey Koren, and Granger Sutton. Assembly algorithms for next-generation sequencing data. *Genomics*, 95(6):315–327, 2010. doi: 10.1016/j.ygeno.2010.03.001.
- [15] Niranjan Nagarajan and Mihai Pop. Sequence assembly demystified. *Nature Reviews Genetics*, 14(3):157–167, 2013. doi: 10.1038/nrg3367.
- [16] Eugene W. Myers. The fragment assembly string graph. *Bioinformatics*, 21(Suppl. 2):ii79–ii85, 2005. doi: 10.1093/bioinformatics/bti1114.
- [17] Pavel A. Pevzner, Haixu Tang, and Michael S. Waterman. An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences*, 98(17):9748–9753, 2001. doi: 10.1073/pnas.171285098.
- [18] Phillip E. C. Compeau, Pavel A. Pevzner, and Glenn Tesler. How to apply de Bruijn graphs to genome assembly. *Nature Biotechnology*, 29(11):987–991, 2011. doi: 10.1038/nbt.2023.
- [19] Daniel R. Zerbino and Ewan Birney. Velvet: algorithms for de novo short read assembly using de Bruijn graphs. *Genome Research*, 18(5):821–829, 2008. doi: 10.1101/gr.074492.107.
- [20] Mikhail Kolmogorov, Jeffrey Yuan, Yu Lin, and Pavel A. Pevzner. Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 37(5):540–546, 2019. doi: 10.1038/s41587-019-0072-8.
- [21] Haoyu Cheng, Gregory T. Concepcion, Xiaowen Feng, Haowen Zhang, and Heng Li. Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, 18: 170–175, 2021. doi: 10.1038/s41592-020-01056-5.
- [22] Mikko Rautiainen, Sergey Nurk, Brian P. Walenz, Glennis A. Logsdon, Daniela Porubsky, Arang Rhie, Evan E. Eichler, Adam M. Phillippy, and Sergey Koren. Telomere-to-telomere assembly of diploid chromosomes with Verkko. *Nature Biotechnology*, 41(10):1474–1482, 2023. doi: 10.1038/s41587-023-01662-6.
- [23] Adrian Fritz, Andreas Bremges, Zhi-Luo Deng, Till Robin Lesker, Jasper Götting, Tina Ganzenmueller, Alexander Sczyrba, Alexander Diltthey, Frank Klawonn, and Alice Carolyn McHardy. Haploflow: strain-resolved de novo assembly of viral genomes. *Genome Biology*, 22(1):212, 2021. doi: 10.1186/s13059-021-02426-8.

- [24] Jordan M. Eizenga, Adam M. Novak, Jonas A. Sibbesen, Simon Heumos, Ali Ghaffaari, Glenn Hickey, Xian Chang, Josiah D. Seaman, Robin Rounthwaite, Jana Ebler, Mikko Rautiainen, Shilpa Garg, Benedict Paten, Tobias Marschall, Jouni Sirén, and Erik Garrison. Pangenome graphs. *Annual Review of Genomics and Human Genetics*, 21:139–162, 2020. doi: 10.1146/annurev-genom-120219-080406.
- [25] Erik Garrison, Andrea Guarracino, Simon Heumos, et al. Building pangenome graphs. *Nature Methods*, 21:2008–2012, 2024. doi: 10.1038/s41592-024-02430-3.
- [26] Jana Ebler, Peter Ebert, William E. Clarke, et al. Pangenome-based genome inference allows efficient and accurate genotyping across a wide spectrum of variant classes. *Nature Genetics*, 54(4):518–525, 2022. doi: 10.1038/s41588-022-01043-w.
- [27] Ghanshyam Chandra, Md Helal Hossen, Stephan Scholz, Alexander T. Diltthey, Daniel Gibney, and Chirag Jain. Integer programming framework for pangenome-based genome inference, 2024.
- [28] Van Hoan Do, Son Hoang Nguyen, Duc Quang Le, et al. Pasa: leveraging population pangenome graph to scaffold prokaryote genome assemblies. *Nucleic Acids Research*, 52(3): e15, 2024. doi: 10.1093/nar/gkad1170.
- [29] Rebecca J. Parsons, Stephanie Forrest, and Christian Burks. Genetic algorithms, operators, and DNA fragment assembly. *Machine Learning*, 21:11–33, 1995. doi: 10.1007/BF00993377.
- [30] Gabriel Luque and Enrique Alba. Metaheuristics for the DNA fragment assembly problem. *International Journal of Computational Intelligence Research*, 1(2):98–108, 2005.
- [31] Enrique Alba and Gabriel Luque. A hybrid genetic algorithm for the DNA fragment assembly problem. In *Recent Advances in Evolutionary Computation for Combinatorial Optimization*, volume 153 of *Studies in Computational Intelligence*, pages 101–112. Springer, 2008. doi: 10.1007/978-3-540-70807-0\_7.
- [32] P. Meksangsouy and Nachol Chaiyaratana. DNA fragment assembly using an ant colony system algorithm. In *Proceedings of the 2003 Congress on Evolutionary Computation*, volume 3, pages 1756–1763. IEEE, 2003.
- [33] Wannasak Wetcharaporn, Nachol Chaiyaratana, and Sissades Tongsimma. DNA fragment assembly: an ant colony system approach. In *Applications of Evolutionary Computing*, volume 3907 of *Lecture Notes in Computer Science*, pages 231–242. Springer, 2006. doi: 10.1007/11732242\_21.
- [34] Guillermo M. Mallén-Fullerton, James Alexander Hughes, Sheridan K. Houghten, and Guillermo Fernández-Anaya. Benchmark datasets for the DNA fragment assembly problem. *International Journal of Bio-Inspired Computation*, 5(6):384–394, 2013. doi: 10.1504/IJBIC.2013.058912.
- [35] Uzma and Zahid Halim. Optimizing the DNA fragment assembly using metaheuristic-based overlap layout consensus approach. *Applied Soft Computing*, 92:106256, 2020. doi: 10.1016/j.asoc.2020.106256.
- [36] Christian Blum. Ant colony optimization: introduction and recent trends. *Physics of Life Reviews*, 2(4):353–373, 2005. doi: 10.1016/j.plrev.2005.10.001.

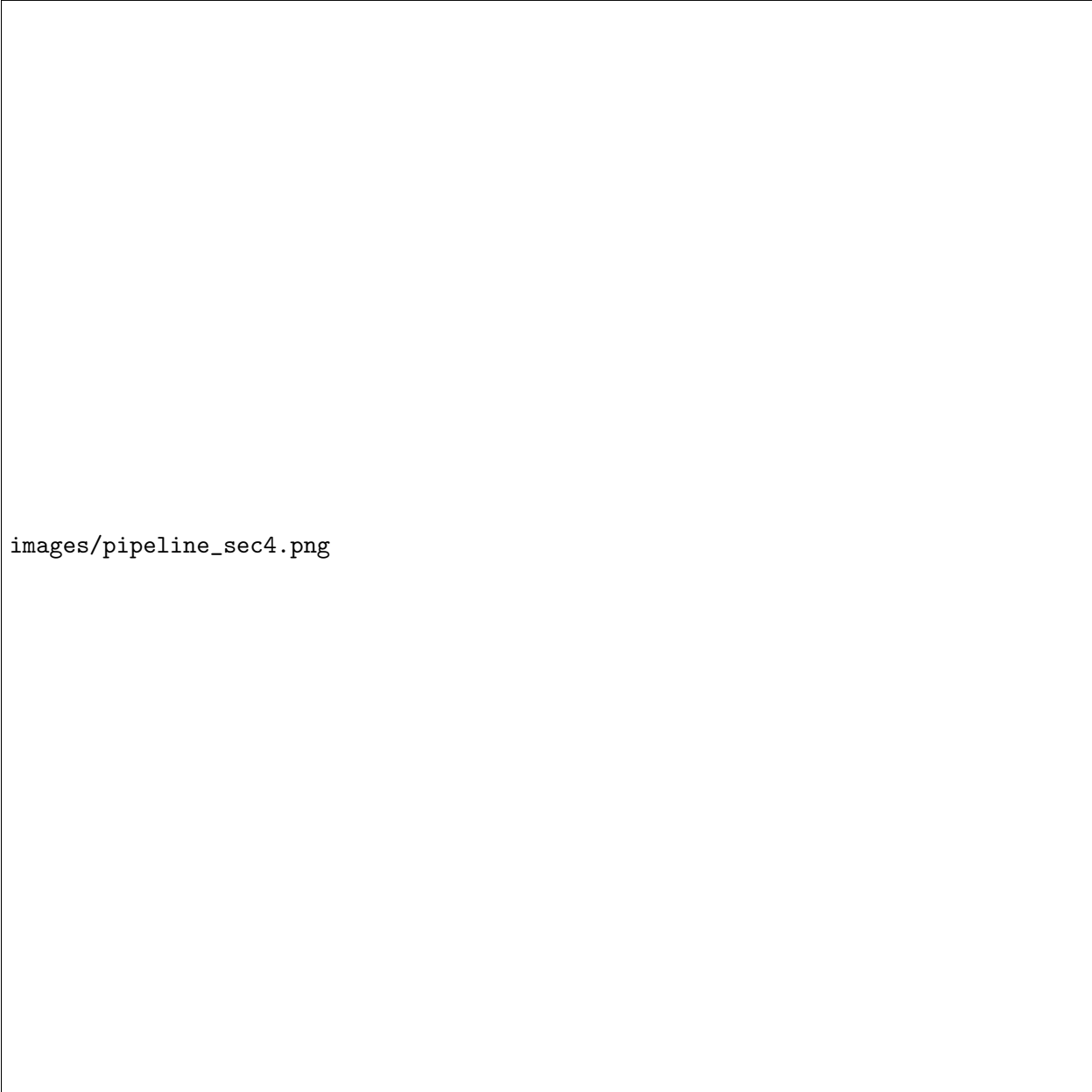
- [37] Lovro Vršek, Xavier Bresson, Thomas Laurent, Martin Schmitz, Kenji Kawaguchi, and Mile Šikić. Geometric deep learning framework for de novo genome assembly. *Genome Research*, 35(4):839–849, 2025. doi: 10.1101/gr.279307.124.
- [38] Lovro Vršek, Xavier Bresson, Thomas Laurent, Martin Schmitz, and Mile Šikić. Learning to untangle genome assembly with graph convolutional networks, 2022.
- [39] Martin Schmitz, Lovro Vršek, Kenji Kawaguchi, and Mile Šikić. DipGNNome: diploid de novo genome assembly with geometric deep learning and beam-search, 2025.
- [40] Kleber Padovani, Rafael Cabral Borges, Roberto Xavier, André Carlos Ponce de Leon Ferreira de Carvalho, Anna Helena Real Costa, Annie Chateau, and Ronnie Alves. Using reinforcement learning in genome assembly: in-depth analysis of a Q-learning assembler. *Frontiers in Bioinformatics*, 5:1633623, 2025. doi: 10.3389/fbinf.2025.1633623.
- [41] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [42] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. In *International Conference on Learning Representations Workshop*, 2017.
- [43] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019.
- [44] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 21188–21198, 2020.
- [45] Jinho Choo, Yeong-Dae Kwon, Jihoon Kim, Jeongwoo Jae, André Hottung, Kevin Tierney, and Youngjune Gwon. Simulation-guided beam search for neural combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 35, pages 8760–8772, 2022.
- [46] Elias B. Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems*, volume 30, pages 6348–6358, 2017.
- [47] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- [48] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [49] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [50] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.
- [51] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International Conference on Computers and Games*, pages 72–83. Springer, 2006.

- [52] David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016. doi: 10.1038/nature16961.
- [53] David Silver, Thomas Hubert, Julian Schrittwieser, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.
- [54] Laurent Orseau and Levi H. S. Lelis. Policy-guided heuristic search with guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 12382–12390, 2021.
- [55] Dat Thanh Tran, Van Khu Vu, and Yining Ma. Beyond static priors: Dynamic neural guidance for large-scale ant colony optimization. In *SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, May 2026.
- [56] Heng Li. Minimap and miniasm: fast mapping and de novo assembly for noisy long sequences. *Bioinformatics*, 32(14):2103–2110, 2016. doi: 10.1093/bioinformatics/btw152.
- [57] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. doi: 10.1007/BF00992696.
- [58] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [59] Bruce T. Lowerre. *The HARPY Speech Recognition System*. PhD thesis, Carnegie Mellon University, 1976.
- [60] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.



images/aco\_pipeline.jpg

Figure 1: Pangenome-guided assembly pipeline with the feasible-walk path-finding module. Graph construction, read simulation, mapping, normalization, and evaluation remain fixed while the path-finding component changes.



images/pipeline\_sec4.png

Figure 2: Learned transition-prior interface for feasible-walk ACO. **Steps 1–3:** The current ant state  $c_t$ , oriented pangenome graph  $\tilde{G}$ , and read-derived copy-number evidence  $w$  are combined as input. **Step 4:** The graph neural network computes edge logits  $z_\theta(e; \tilde{G}, w)$  *once* per outer iteration as a held prior. **Steps 5–6:** The transition score  $\log s(e | c_t) = \alpha \log \tau_e + \beta \log \eta(v, c_t) + \gamma r_\theta(e; \tilde{G}, w)$  combines pheromone, residual-copy heuristic, and neural prior to rank legal outgoing edges. **Step 7:** Inner ACO rollouts sample feasible walks, evaluate node-copy energy  $J(W_a)$ , update pheromone via evaporation and elite deposit (Eq. 16), and store trajectory snapshots  $\tau^{(h,t)}$ . **Step 8:** Online policy-gradient training replays stored trajectories using the saved  $\tau$  snapshot, computes batch-centered advantages  $A_a$ , minimizes the REINFORCE loss  $\mathcal{L}(\theta)$  [57], and updates the graph neural network.